

**Carrera:**  
**Programación y WebMaster**

**Docente:**  
**MTRO. Mario Alonso Segovia Gutiérrez**

**Materia:**  
**Proyecto de una Plataforma Virtual**

**Alumno(s):**  
**Salomon Campos Pablo Leonel**

**Fecha:**  
**03 de Julio 2026**

## Introducción

En el presente documento se analiza el caso práctico del proyecto SmartLibrary, que es una plataforma web que sirve para administrar una biblioteca digital. En el caso que se estará evaluando el equipo ha compartido carpetas en la nube etc etc. Durante el desarrollo del proyecto surgieron problemas y/o conflictos en trabajos sobreescritos etc etc, a partir de esta situación la propuesta es una forma de trabajar usando Git, con el objetivo de mejorar la calidad del proyecto.



## **Parte 1. Análisis del problema**

### **1. ¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?**

Unos de los principales errores fue trabajar sin una estrategia para administrar el código fuente. El equipo de desarrollo usaba una carpeta en la nube y cada desarrollador reemplazaba el proyecto completo con su versión, sin controlar qué archivos habían cambiado ni quién los modificó. También faltó una forma de integrar avances, revisar cambios y conservar versiones anteriores. Esto provocó que algunas funciones correctas se perdieran o fueran reemplazadas accidentalmente por versiones incompletas.

### **2. ¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?**

Compartir carpetas puede servir para documentos, pero no es adecuado para proyectos de software. Cuando varias personas editan archivos al mismo tiempo, la carpeta no entiende la lógica del código ni puede combinar cambios de forma segura. Si alguien sube una copia vieja del proyecto, puede borrar avances recientes.

### **3. ¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?**

En proyectos grandes, esta forma de trabajo puede causar retrasos, errores y pérdida de confianza en el sistema. También puede afectar la entrega al cliente, porque integrar cambios sin control aumenta el riesgo de que una corrección dañe otra parte del proyecto. En un sistema más grande, con muchos programadores, el desorden puede hacer que el equipo pierda demasiado tiempo investigando problemas que se pudieron haber evitado desde un principio al usar un control de versiones.

## **Parte 2. Importancia del control de versiones**

### **1. ¿Qué es un sistema de control de versiones?**

Un sistema de control de versiones es una herramienta que permite registrar, organizar y administrar los cambios realizados en los archivos de un proyecto. En programación, se utiliza principalmente para llevar el historial del código fuente. Esto permite saber qué se cambió, cuándo se cambió y quién realizó cada modificación. Git es uno de los sistemas más utilizados porque permite trabajar de forma local y colaborar con otros desarrolladores mediante repositorios remotos.

### **2. ¿Cuál es su principal objetivo?**

Su principal objetivo es proteger el avance del proyecto y permitir que el equipo trabaje de manera ordenada. Gracias al control de versiones, los cambios no se mezclan de forma improvisada, sino que se integran mediante un proceso más seguro. También permite recuperar versiones anteriores si algo falla y comparar modificaciones para entender mejor el comportamiento del sistema.

### **3. ¿Qué ventajas ofrece frente al manejo tradicional de archivos?**

A diferencia de guardar carpetas con nombres como “final”, “final corregido” o “versión nueva”, el control de versiones conserva un historial real de cambios. También permite que varios desarrolladores trabajen en paralelo sin sobrescribir el trabajo de los demás. Otra ventaja es que facilita la detección de errores, la revisión del código, la creación de ramas para nuevas funciones y la integración gradual de avances al proyecto principal.

## Parte 3. Propuesta de la organización

### 1. ¿Qué rama utilizarías para la versión estable del sistema?

Utilizaría la rama main para la versión estable. Esta rama es el estado más confiable del proyecto, es decir, la versión que ya fue probada y que podría entregarse o mostrarse al cliente sin afectar el funcionamiento del sistema.

### 2. ¿Qué rama utilizarías para integrar el trabajo del equipo?

Usaría la rama develop para integrar el trabajo del equipo. En esta rama se juntarían los avances del proyecto como usuarios, préstamos, reportes y correcciones etc etc. Antes de pasar a main, el equipo podría hacer pruebas generales en develop para verificar que los módulos funcionen correctamente sin afectar la rama principal main.

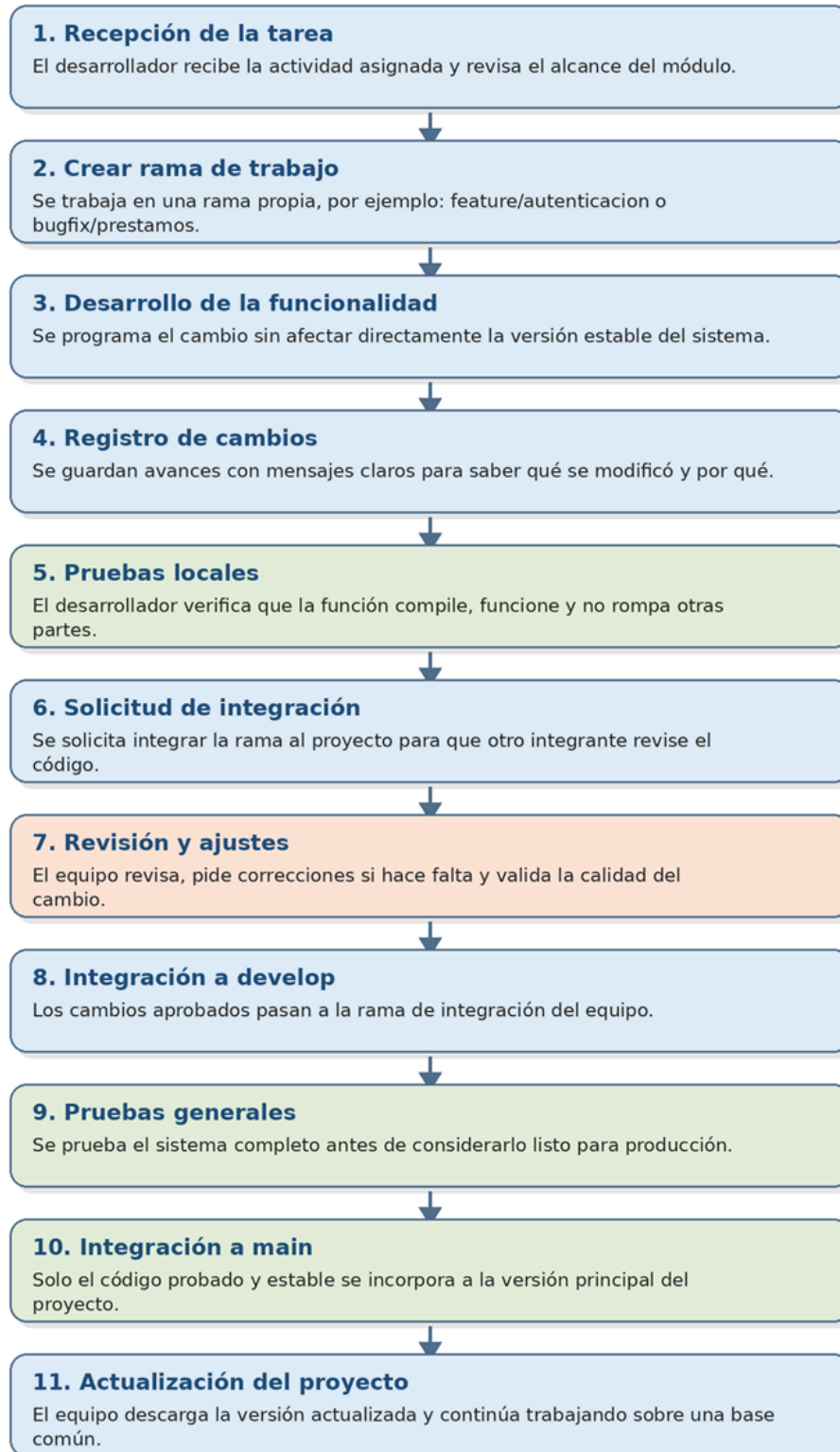
### 3. ¿Cada desarrollador debería trabajar directamente sobre la rama principal? Explica tu respuesta.

No, ya que cada desarrollador debería trabajar en una rama independiente según la tarea asignada ya que trabajar directamente sobre main es un riesgo, porque cualquier error quedaría dentro de la versión estable. Al usar ramas, cada integrante puede desarrollar, probar y corregir sin afectar el avance de los demás

### 4. ¿Cuándo deberían integrarse los cambios al proyecto?

Los cambios deberían integrarse cuando la tarea asignada esté terminada, probada y revisada por al menos otro integrante del equipo. También es importante que el desarrollador actualice su rama con la versión más reciente de develop antes de solicitar la integración, para reducir conflictos y detectar errores al subir sus cambios.

## Parte 4. Flujo de trabajo





## **Parte 5. Análisis de situaciones**

### **1. ¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?**

Si dos desarrolladores modifican el mismo archivo al mismo tiempo, puede aparecer un conflicto al intentar integrar los cambios, ya que Git ayuda a identificar qué partes del archivo fueron modificadas por cada persona y decidir qué cambios conservar y ajustar el archivo para que quede una versión estable.

### **2. ¿Por qué es importante conservar un historial de cambios en un proyecto?**

El historial de cambios es importante porque sirve para saber quién hizo una modificación, en qué fecha y con qué propósito. También ayuda a regresar a una versión anterior si una actualización causa errores.

### **3. ¿Qué beneficios aporta trabajar mediante ramas independientes?**

Las ramas independientes permiten separar el trabajo de cada desarrollador. De esta manera, un módulo incompleto no afecta a todo el sistema. También facilitan la revisión del código, la prueba de nuevas funciones y la corrección de errores.

### **4. ¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?**

Cuando aparece un error después de una actualización, el control de versiones permite revisar qué cambios se integraron recientemente. El equipo puede comparar versiones, identificar el cambio que causó el problema y, si es necesario, regresar temporalmente a una versión anterior.

### **5. ¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?**

Es indispensable porque permite coordinar el trabajo de varias personas sobre un mismo proyecto y llevar un control del código. Sin esta herramienta, el equipo depende de copias manuales y corre el riesgo de perder información, con Git, cada programador puede aportar cambios de forma controlada, revisar el trabajo de otros etc etc.

## Conclusión

Git se ha convertido en una herramienta importante en la programación, porque permite trabajar de manera colaborativa, nos permite crear ramas, registrar cambios y recuperar versiones anteriores, además, se adapta a equipos pequeños y grandes, ya que cada integrante puede trabajar en una parte del sistema en ramas diferentes sin afectar el sistema y versiones que sean estables.

Una buena organización del código reduce errores, evita pérdidas de información y facilita la integración de nuevas funcionalidades.